

Here is the step-by-step setup.

Step 1: Create the GitHub repo

Create a new private repo on GitHub called airis (or whatever you prefer). Then clone it locally and copy in everything we have built:

```
airis/
├── ARCHITECTURE.md
├── CLAUDE.md
├── design-system/
│   ├── AIRIS_Design-System_v1.md
│   ├── AIRIS_Component-Library_v1.html
│   └── tailwind.config.js
├── docs/
│   └── AIRIS_PRD_v1.md
├── services/
│   ├── auth/SPEC.md
│   ├── candidate-management/SPEC.md
│   ├── job-management/SPEC.md
│   ├── pipeline/SPEC.md
│   ├── scheduling/SPEC.md
│   ├── communication/SPEC.md
│   ├── ai-services/SPEC.md
│   └── analytics/SPEC.md
```

Commit this as your initial commit. This is your 'contracts' commit: everything the team builds against.

Step 2: Set up the Next.js frontend project

One developer (or you via Claude Code) initialises the project:

```
npx create-next-app@latest airis-app --typescript --tailwind
--eslint --app --src-dir
cd airis-app
npx shadcn@latest init
```

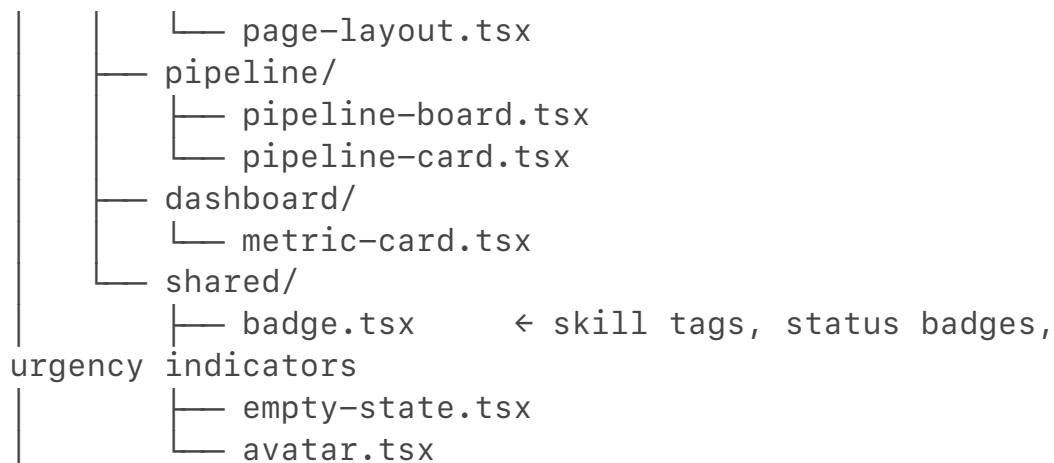
During shadcn init, select the 'New York' style and pick slate as the base colour (we override it anyway). Then replace the generated tailwind.config.ts with the one from design-system/tailwind.config.js. Install Inter from Google Fonts in your layout file.

Commit this scaffolding to the repo.

Step 3: Create a shared component library inside the repo

Create a folder at src/components/ui/ (shadcn already does this). Then add your AIRIS-specific shared components alongside it:

```
src/
├── components/
│   ├── ui/                ← shadcn/ui primitives (button,
│   │   │   input, table, dialog, etc.)
│   ├── layout/
│   │   ├── sidebar.tsx
│   │   └── topbar.tsx
```



The key rule: all developers import from `@/components/`. Nobody creates their own button or card. If a component does not exist yet, they add it here, following the design system spec.

Step 4: Protect the design system with branch rules

In GitHub repo settings, go to Branches and add a rule for main:

- Require pull request reviews before merging (at least 1 reviewer)
- Require status checks to pass (once you add CI)

Then add a CODEOWNERS file at the repo root:

```
# CODEOWNERS
/design-system/           @shen
/src/components/ui/       @shen
/tailwind.config.*        @shen
/ARCHITECTURE.md          @shen
```

Replace @shen with your GitHub username. This means any change to the design system, shared components, or Tailwind config requires your approval. Developers can build feature pages freely, but they cannot change the foundation without your sign-off.

Step 5: Add a linting rule to enforce token usage

Install the Tailwind CSS IntelliSense extension for VS Code (your team probably already has it). Then add an ESLint rule to catch hardcoded colours. Create or update `.eslintrc.js`:

```
module.exports = {
  // ...existing config
  rules: {
    'no-restricted-syntax': [
      'error',
      {
        selector: 'Literal[value=/^#[0-9a-fA-F]{3,8}$/',
        message: 'Do not use hardcoded hex colours. Use Tailwind tokens from the design system (e.g. text-primary-500).',
      },
    ],
  },
},
```

```
};
```

This catches developers who write `style={{ color: '#28808F' }}` instead of using the Tailwind class.

Step 6: Add the component library page to the repo

Move the HTML component library to a place the team can access it easily. Two options:

Option A (simplest): keep it in `design-system/` and tell the team to open it locally in a browser. Any developer clones the repo and opens `design-system/AIRIS_Component-Library_v1.html` directly.

Option B (better for remote teams): deploy it as a GitHub Pages site. Add a workflow at `.github/workflows/design-system.yml`:

```
name: Deploy design system
on:
  push:
    branches: [main]
    paths: ['design-system/**']
jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      pages: write
      id-token: write
    environment:
      name: github-pages
    steps:
      - uses: actions/checkout@v4
      - uses: actions/upload-pages-artifact@v3
        with:
          path: design-system
      - uses: actions/deploy-pages@v4
```

Enable GitHub Pages in repo settings (source: GitHub Actions). The component library is then live at https://your-org.github.io/airis/AIRIS_Component-Library_v1.html. Bookmark it, share it in Slack.

Step 7: Structure developer workflow

Each developer works on one service at a time. The workflow is:

1. Create a feature branch: `git checkout -b feature/auth-service`
2. Read the service's `SPEC.md` and the design system spec
3. Build the backend (following the SPEC) and frontend pages (following the design system)
4. Import shared components from `@/components/`, never build custom versions
5. Open a PR. CODEOWNERS auto-requests your review if they touched shared files.
6. Merge after review.

Step 8: Add a CLAUDE.md for AI-assisted development

The CLAUDE.md at the repo root should say:

```
# AIRIS
```

Read ARCHITECTURE.md first for system orientation.

Each service has a SPEC.md with full contracts and test cases.

All UI must follow design-system/AIRIS_Design-System_v1.md.

Use only Tailwind tokens from tailwind.config.js. No hardcoded colours.

Import shared components from @/components/. Do not create custom

buttons, inputs, tables, badges, or cards.

This ensures that whether a developer uses Claude Code, Cursor, or Copilot, the AI tool reads the same constraints.

That is the full setup. The design system is protected by CODEOWNERS, enforced by Tailwind tokens and ESLint, visible via the HTML reference page, and documented in the spec. Developers can build their assigned services independently while the UI stays consistent.